

Game Playing

Professor Dr Mohammad Shahadat Hossain

Department of Computer Science &

Engineering

University of Chittagong



The Game

- In which we examine the problems that arise when we try to plan ahead in a world that includes a hostile agent
 - The presence of an opponent makes the decision problem somewhat more complicated than the search problems.
 - A game can be defined by the initial state (how the board is set up), the operators (which define the legal moves), a terminal test (which says when the game is over), and a utility or payoff function (which says who won, and by how much)
- 

Why study games

- Fun
- Clear criteria for success
- Offer an opportunity to study problems involving {hostile, adversarial, competing} agents.
- Interesting, hard problems which require minimal “initial structure”
- Games often define very large search spaces
 - chess 35^{100} nodes in search tree, 10^{40} legal states
- Historical reasons
- Different from games studied in game theory

Typical Case (perfect games)

- 2-person game
- Players alternate moves
- Zero-sum-- one player's loss is the other's gain.
- Perfect information -- both players have access to complete information about the state of the game. No information is hidden from either player.
- No chance (e.g., using dice) involved
- Clear rules for legal moves (no uncertain position transition involved)
- Well-defined outcomes (W/L/D)
- Examples: Tic-Tac-Toe, Checkers, Chess, Go, Nim, Othello
- Not: Bridge, Solitaire, Backgammon, ...

How to play a game

- A way to play such a game is to:
 - Consider all the legal moves you can make.
 - Each move leads to a new board configuration (position).
 - Evaluate each resulting position and determine which is best
 - Make that move.
 - Wait for your opponent to move and repeat?
 - Key problems are:
 - Representing the “board”
 - Generating all legal next boards
 - Evaluating a position
 - Look ahead
- 

Game Trees

- Problem spaces for typical games represented as trees.
- Root node represents the “board” configuration at which a decision must be made as to what is the **best single move to make next**. (not necessarily the initial configuration)
- **Evaluator function** rates a board position. $f(\text{board})$ (a real number).
- Arcs represent the possible legal moves for a player (no costs associated to arcs)
- Terminal nodes represent end-game configurations (the result must be one of “win”, “lose”, and “draw”, possibly with numerical payoff)

- If it is my turn to move, then the root is labeled a "MAX" node; otherwise it is labeled a "MIN" node indicating my opponent's turn.
- Each level of the tree has nodes that are all MAX or all MIN; nodes at level i are of the opposite kind from those at level $i+1$
- Complete game tree: includes all configurations that can be generated from the root by legal moves (all leaves are terminal nodes)
- Incomplete game tree: includes all configurations that can be generated from the root by legal moves to a given depth (looking ahead to a given steps)

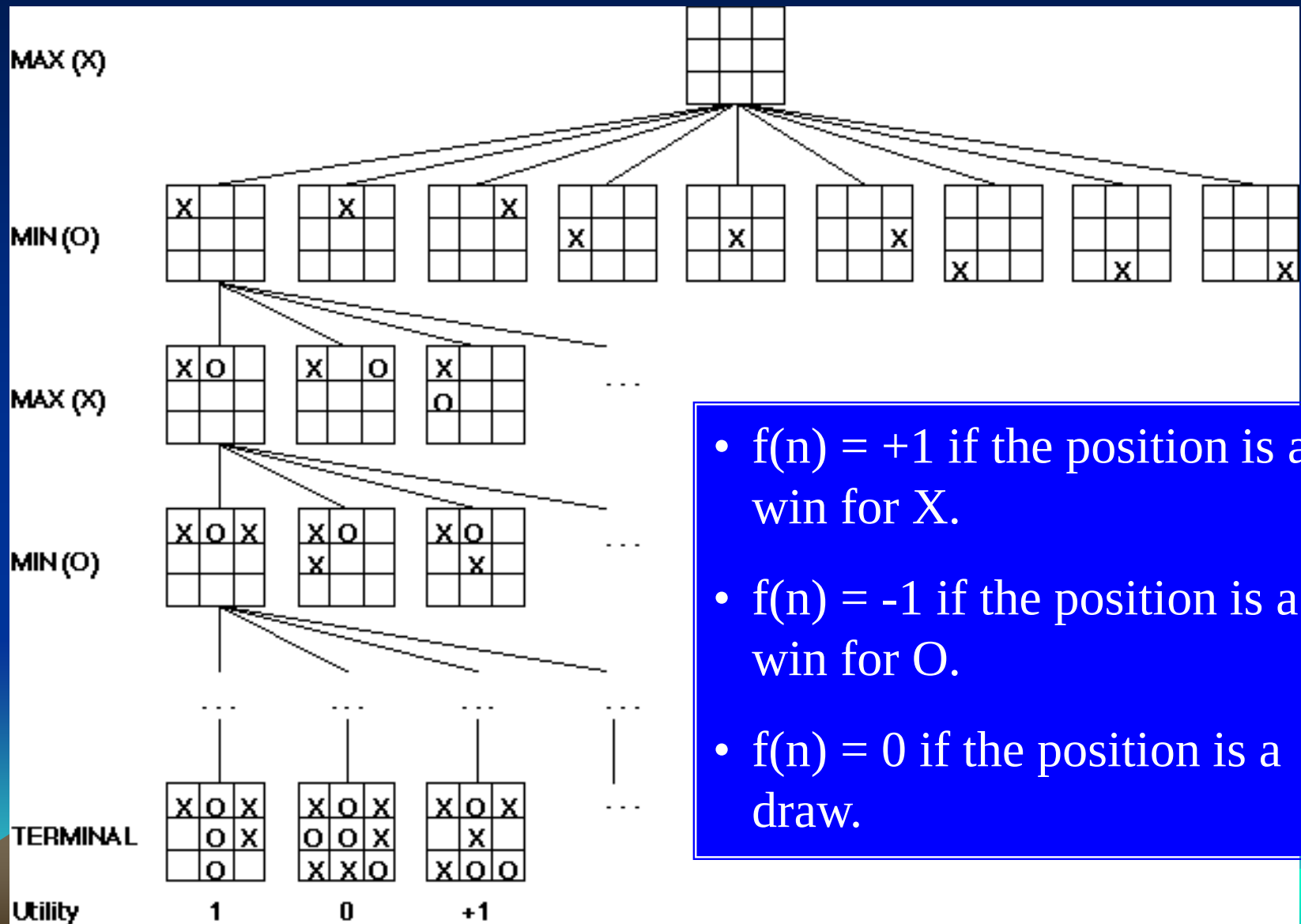
Evaluation Function

- **Evaluation function** or **static evaluator** is used to evaluate the "goodness" of a game position.
 - Contrast with heuristic search where the evaluation function was a non-negative estimate of the cost from the start node to a goal and passing through the given node.
- The zero-sum assumption allows us to use a single evaluation function to describe the goodness of a board with respect to both players.
 - $f(n) > 0$: position n good for me and bad for you.
 - $f(n) < 0$: position n bad for me and good for you
 - $f(n)$ near 0: position n is a neutral position.
 - $f(n) \gg 0$: win for me.
 - $f(n) \ll 0$: win for you..

- Evaluation function is a heuristic function, and it is where the domain experts' knowledge resides.
- Example of an Evaluation Function for Tic-Tac-Toe:
$$f(n) = [\text{\# of 3-lengths open for me}] - [\text{\# of 3-lengths open for you}]$$

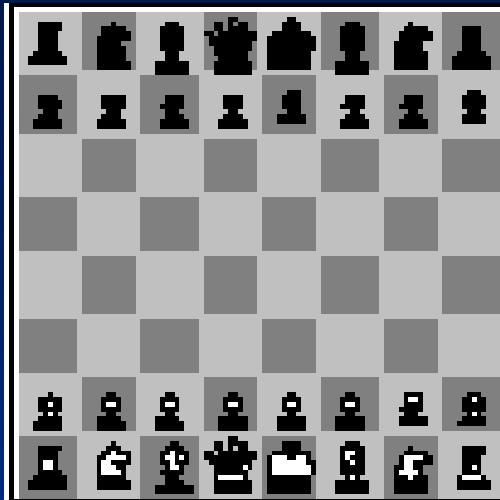
where a 3-length is a complete row, column, or diagonal.
- Alan Turing's function for chess
 - $f(n) = w(n)/b(n)$ where $w(n)$ = sum of the point value of white's pieces and $b(n)$ is sum for black.
- Most evaluation functions are specified as a weighted sum of position features:
$$f(n) = w_1 \cdot \text{feat}_1(n) + w_2 \cdot \text{feat}_2(n) + \dots + w_n \cdot \text{feat}_k(n)$$
- Example features for chess are piece count, piece placement, squares controlled, etc.
- Deep Blue has about 6,000 features in its evaluation function.

An example (partial) game tree for Tic-Tac-Toe

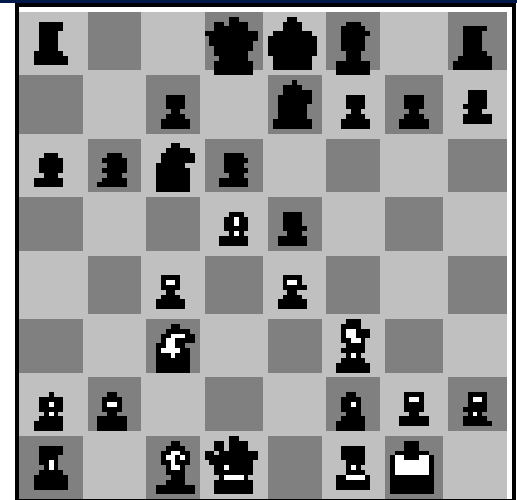


- $f(n) = +1$ if the position is a win for X.
- $f(n) = -1$ if the position is a win for O.
- $f(n) = 0$ if the position is a draw.

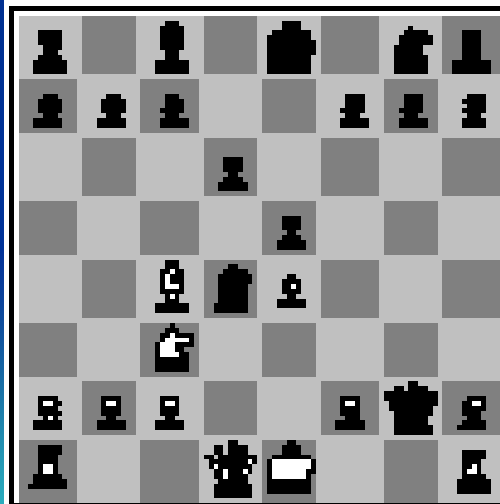
Some Chess Positions and their Evaluations



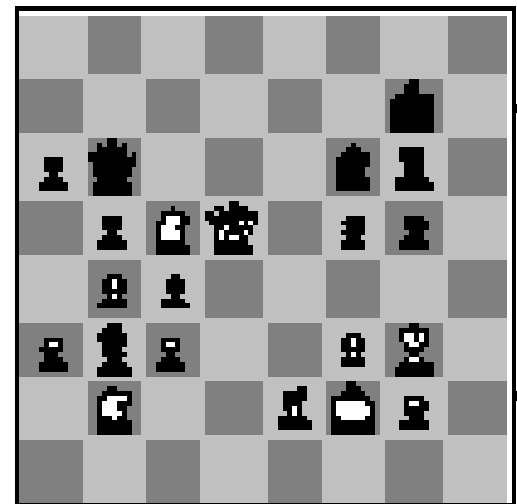
(a) 100% to a loss
34% down



(b) 34% to a loss
100% slightly better



(c) 100% to a loss
34% winning



(d) 34% to a loss
100% about to lose

In two-player games with perfect information, the minmax algorithm can determine the best move for a player (assuming the opponent plays perfectly) by determining the entire game tree



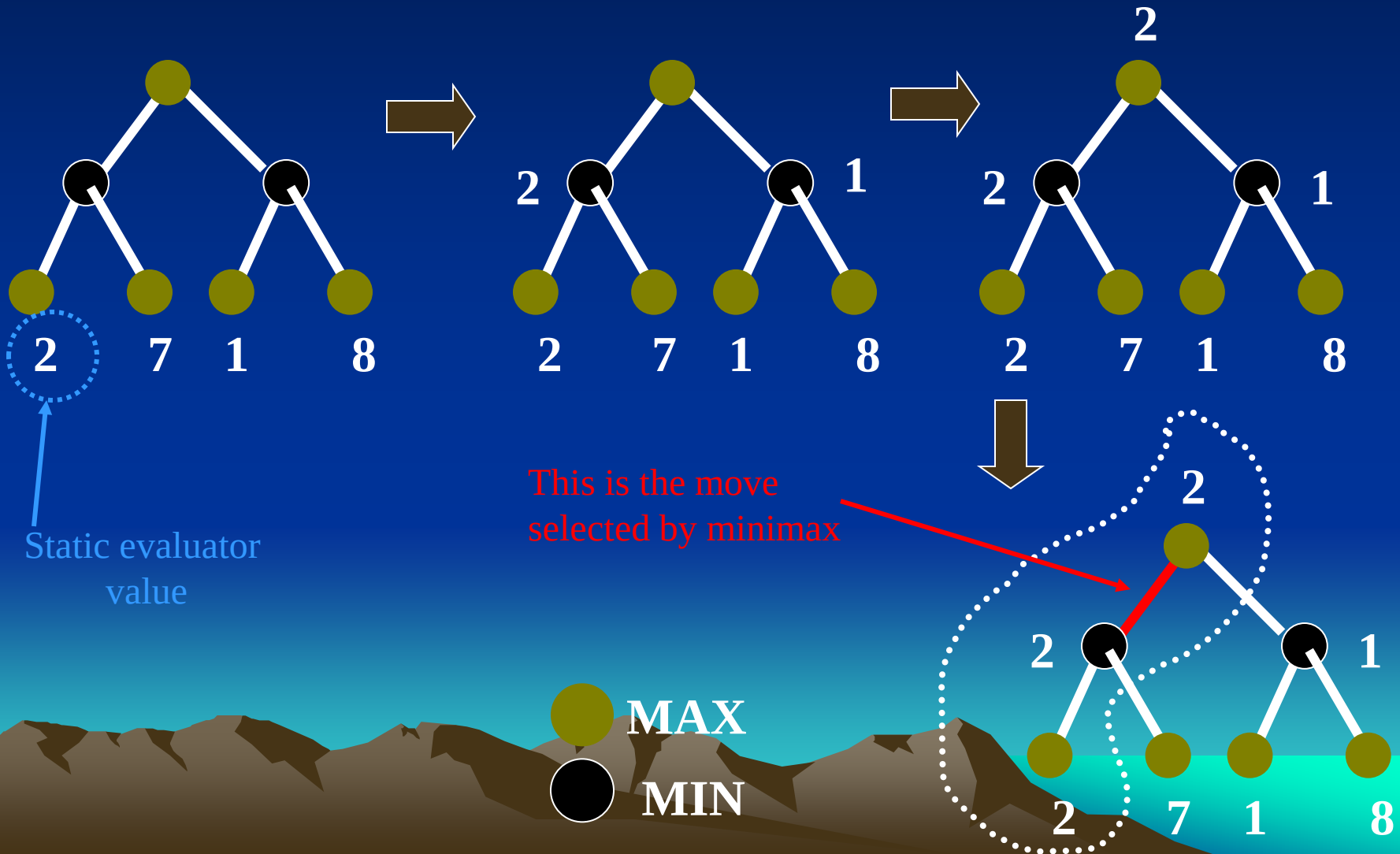
Minimax Rule

- Goal of game tree search: to determine **one move** for Max player that **maximizes** the **guaranteed payoff** for a given game tree for MAX
 - Regardless of the moves the MIN will take
 - The value of each node (Max and MIN) is determined by (back up from) the values of its children
 - MAX plays the worst case scenario:
 - Always assume MIN to take moves to maximize his pay-off (i.e., to minimize the pay-off of MAX)
 - For a MAX node, the backed up value is the **maximum** of the values associated with its children
 - For a MIN node, the backed up value is the **minimum** of the values associated with its children
- 

Minimax procedure

- Create start node as a MAX node with current board configuration
- Expand nodes down to some depth (i.e., ply) of lookahead in the game.
- Apply the evaluation function at each of the leaf nodes
- Obtain the “back up” values for each of the non-leaf nodes from its children by Minimax rule until a value is computed for the root node.
- Pick the operator associated with the child node whose backed up value determined the value at the root as the move for MAX

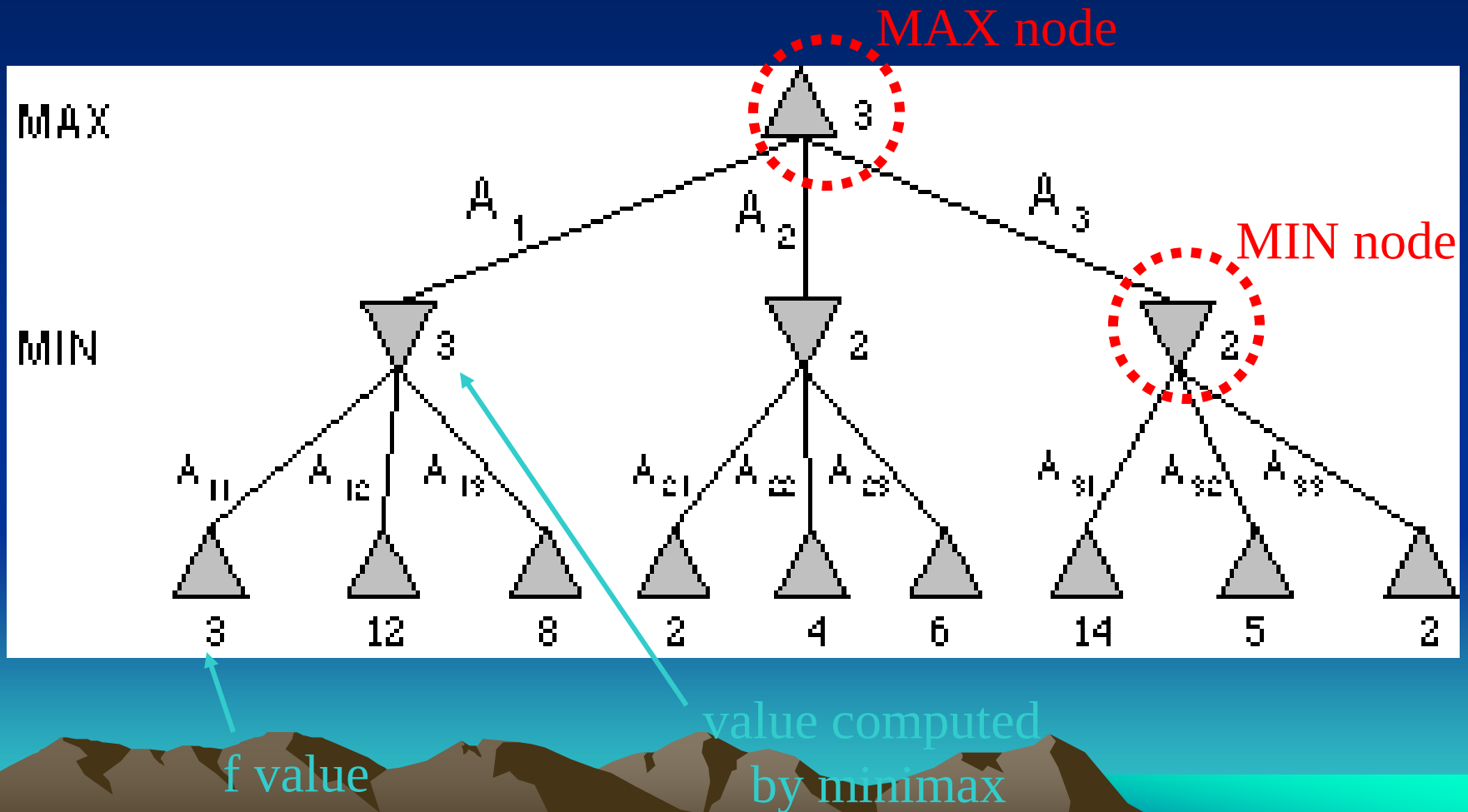
Minimax Search



Comments on Minimax search

- The search is **depth-first** with the given depth (ply) as the limit
 - Time complexity: $O(b^d)$
 - Linear space complexity
- Performance depends on
 - Quality of evaluation functions (domain knowledge)
 - Depth of the search (computer power and search algorithm)
- Different from ordinary state space search
 - Not to search for a solution but for one move only
 - No cost is associated with each arc
 - MAX does not know how MIN is going to counter each of his moves
- Minimax rule is a basis for other game tree search algorithms

Minimax Tree

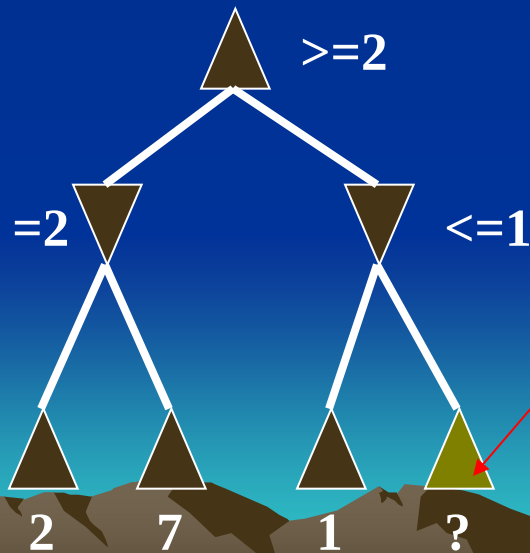


The alpha-beta algorithm does the same calculation as Minmax but is more efficient because it prunes (allows us to ignore portions of the search tree that make no difference to the final choice) away branches of the search tree that it can prove are irrelevant to the final outcome.



Alpha-beta pruning

- We can improve on the performance of the minimax algorithm through alpha-beta pruning.
- Basic idea: *"If you have an idea that is surely bad, don't take the time to see how truly awful it is."* -- Pat Winston



- We don't need to compute the value at this node.
- No matter what it is it can't effect the value of the root node.

Alpha-beta pruning

- Traverse the search tree in depth-first order
 - At each **Max** node n , **alpha**(n) = maximum value found so far
 - Start with $-\infty$ and only increase
 - Increases if a child of n returns a value greater than the current alpha
 - Serve as a tentative lower bound of the final pay-off
 - At each **Min** node n , **beta**(n) = minimum value found so far
 - Start with ∞ and only decrease
 - Decreases if a child of n returns a value less than the current beta
 - Serve as a tentative upper bound of the final pay-off
 - **beta**(n) for **MAX** node n : smallest beta value of its MIN ancestors
 - **alpha**(n) for **MIN** node n : greatest alpha value of its MAX ancestors
- 

Alpha-beta pruning

- Carry alpha and beta values down during search
 - alpha can be changed only at MAX nodes
 - beta can be changed only at MIN nodes
 - Pruning occurs whenever $\alpha \geq \beta$
- **alpha cutoff:**
 - Given a Max node n , cutoff the search below n (i.e., don't generate any more of n 's children) **if $\alpha(n) \geq \beta(n)$**
(alpha increases and passes beta from below)
- **beta cutoff:**
 - Given a Min node n , cutoff the search below n (i.e., don't generate any more of n 's children) **if $\beta(n) \leq \alpha(n)$**
(beta decreases and passes alpha from above)

Alpha-beta algorithm

initiation :

$\alpha := -\infty$

$\beta := \infty$

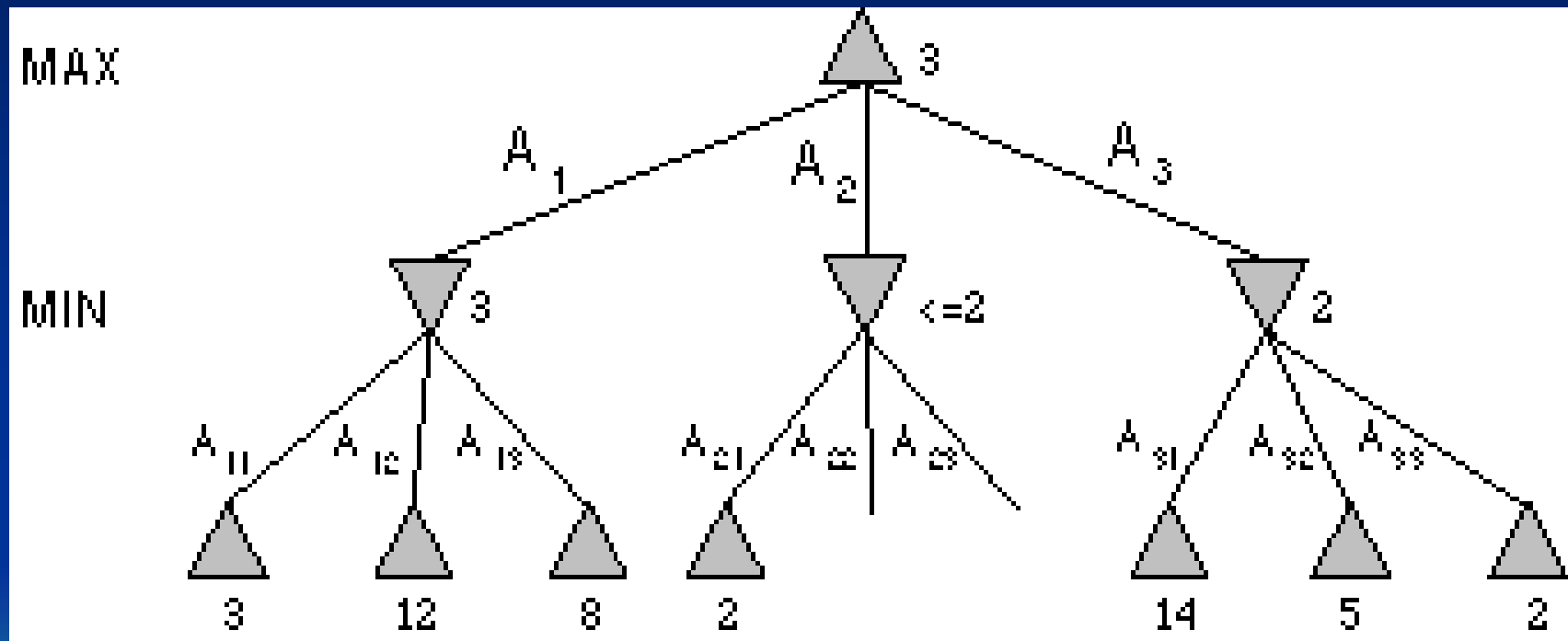
- Two functions recursively call each other

```
function MAX-value (n, alpha, beta)
  if n is a leaf node then return f(n);
  for each child n' of n do
    alpha := max{alpha, MIN-value(n', alpha, beta)};
    if alpha >= beta then return beta /* pruning */
  end{do}
  return alpha
```

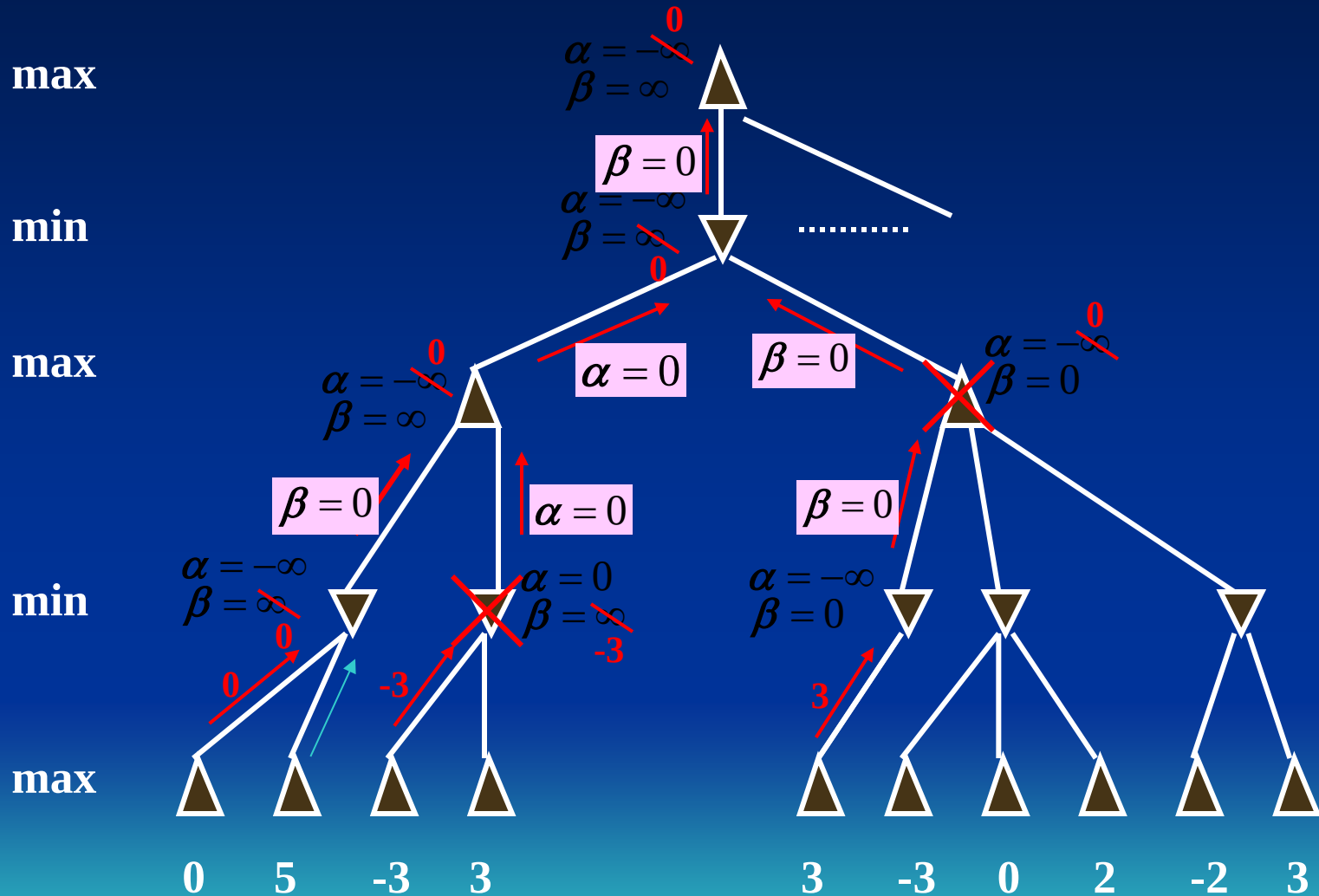
```
function MIN-value (n, alpha, beta)
  if n is a leaf node then return f(n);
  for each child n' of n do
    beta := min{beta, MAX-value(n', alpha, beta)};
    if beta <= alpha then return alpha /* pruning */
  end{do}
  return beta
```



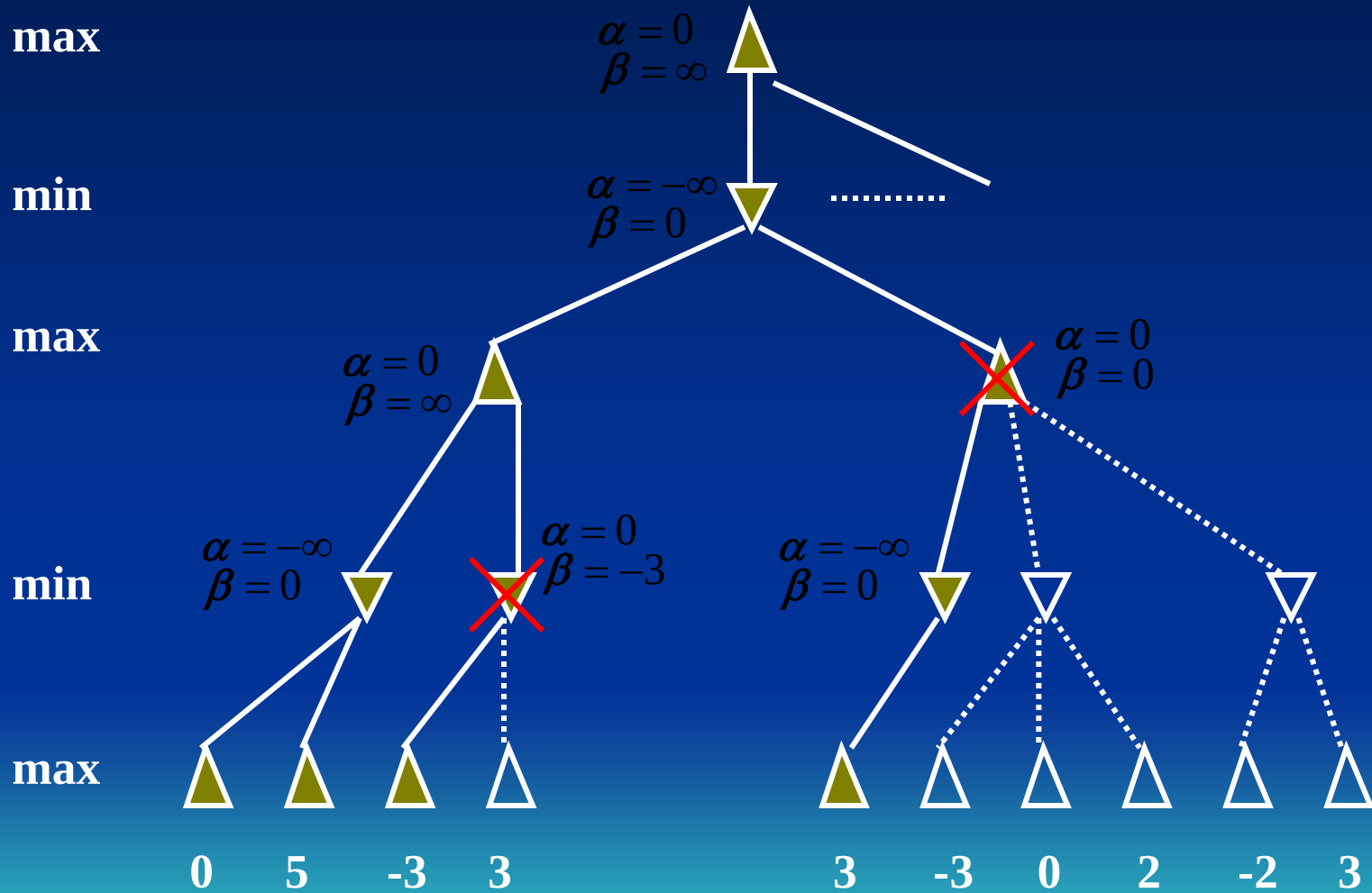
Alpha-beta search



An example of Alpha-beta pruning



Final tree



Effectiveness of Alpha-beta pruning

- Alpha-Beta is guaranteed to compute the same value for the root node as computed by Minimax.
- **Worst case:** NO pruning, examining $O(b^d)$ leaf nodes, where each node has b children and a d -ply search is performed
- **Best case:** examine only $O(b^{(d/2)})$ leaf nodes.
 - You can search twice as deep as Minimax! Or the branch factor is $b^{(1/2)}$ rather than b .
- **Best case** is when each player's best move is the leftmost alternative, i.e. at MAX nodes the child with the largest value generated first, and at MIN nodes the child with the smallest value generated first.
- In Deep Blue, they found empirically that Alpha-Beta pruning meant that the average branching factor at each node was about 6 instead of about 35-40

Assignment

- Read Section 5.1- 5.6

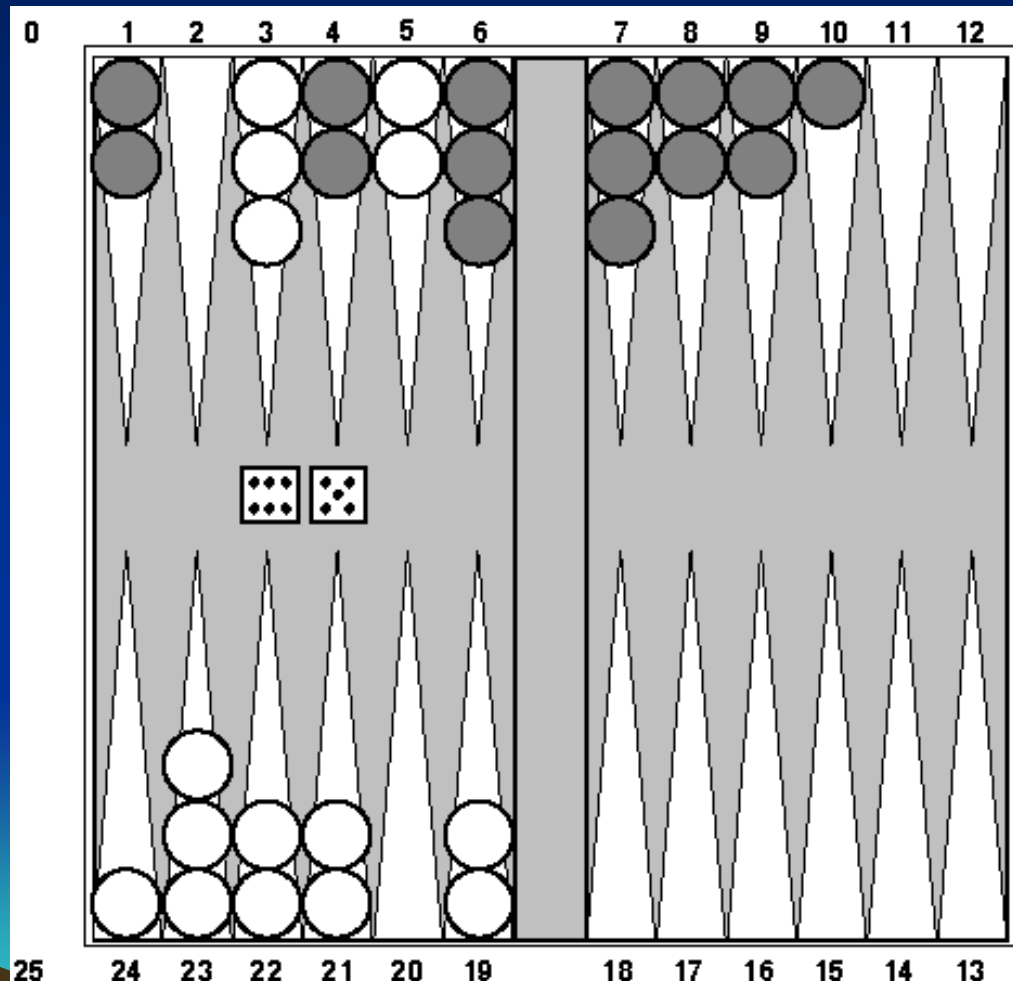


Optional



Games of Chance

- Backgammon is a two player game with **uncertainty**.
- Players roll dice to determine what moves to make.
- White has just rolled *5 and 6* and had four legal moves:
 - 5-10, 5-11
 - 5-11, 19-24
 - 5-10, 10-16
 - 5-11, 11-16
- Such games are good for exploring decision making in adversarial problems involving skill and luck.



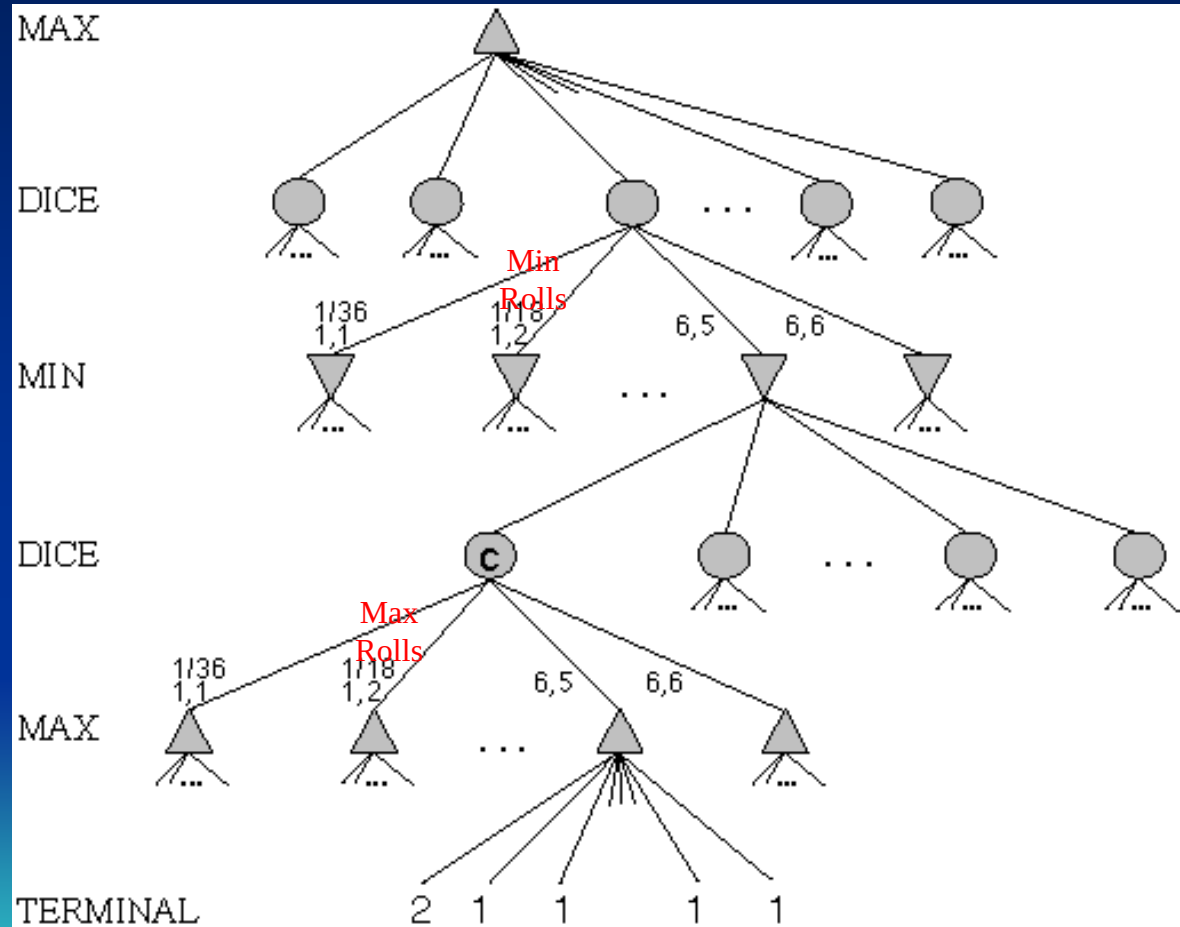
Game Trees with Chance Nodes

- **Chance nodes** (shown as circles) represent the dice rolls.
- Each chance node has 21 distinct children with a probability associated with each.
- We can use minimax to compute the values for the MAX and MIN nodes.
- Use **expected values** for chance nodes.
- For chance nodes over a max node, as in C, we compute:

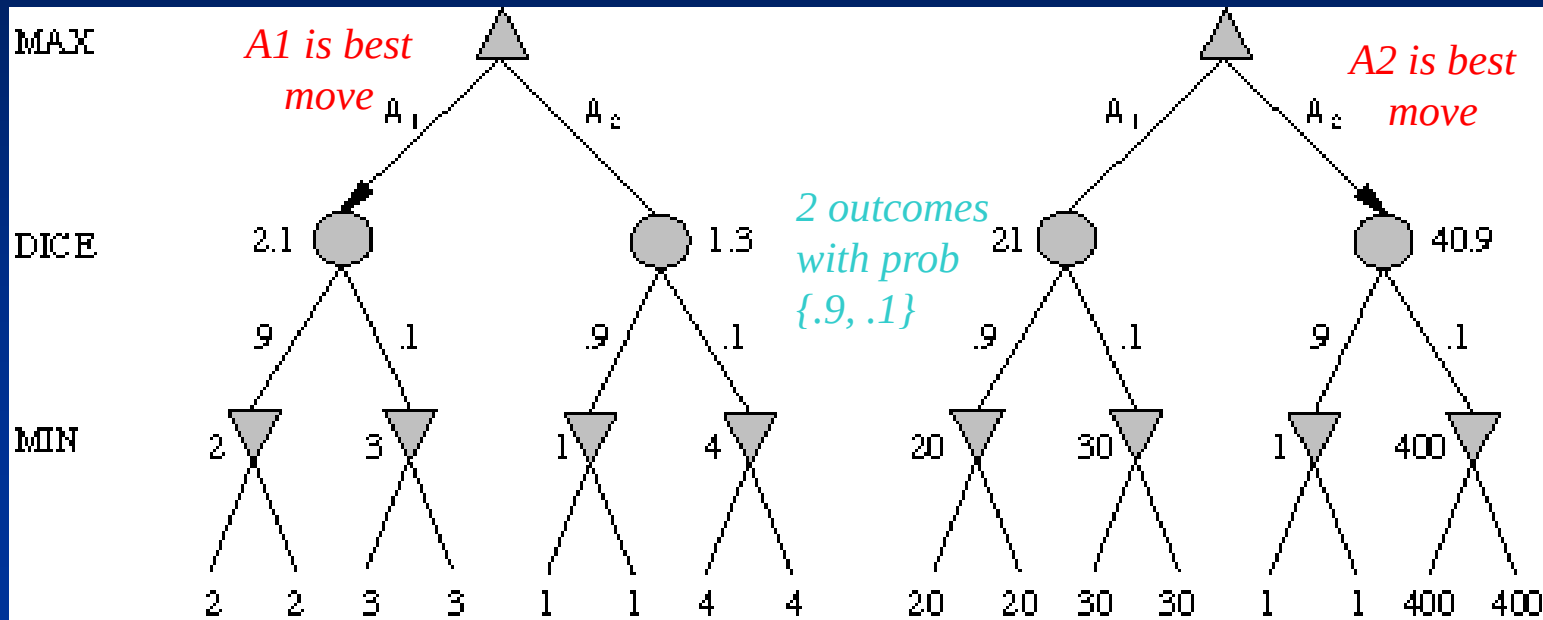
$$\text{expectimax}(C) = \text{Sum}_j (P(d_j) * \text{maxvalue}(i))$$

- For chance nodes over a min node compute:

$$\text{expectimin}(C) = \sum_j (P(d_j) * \text{minvalue}(i))$$

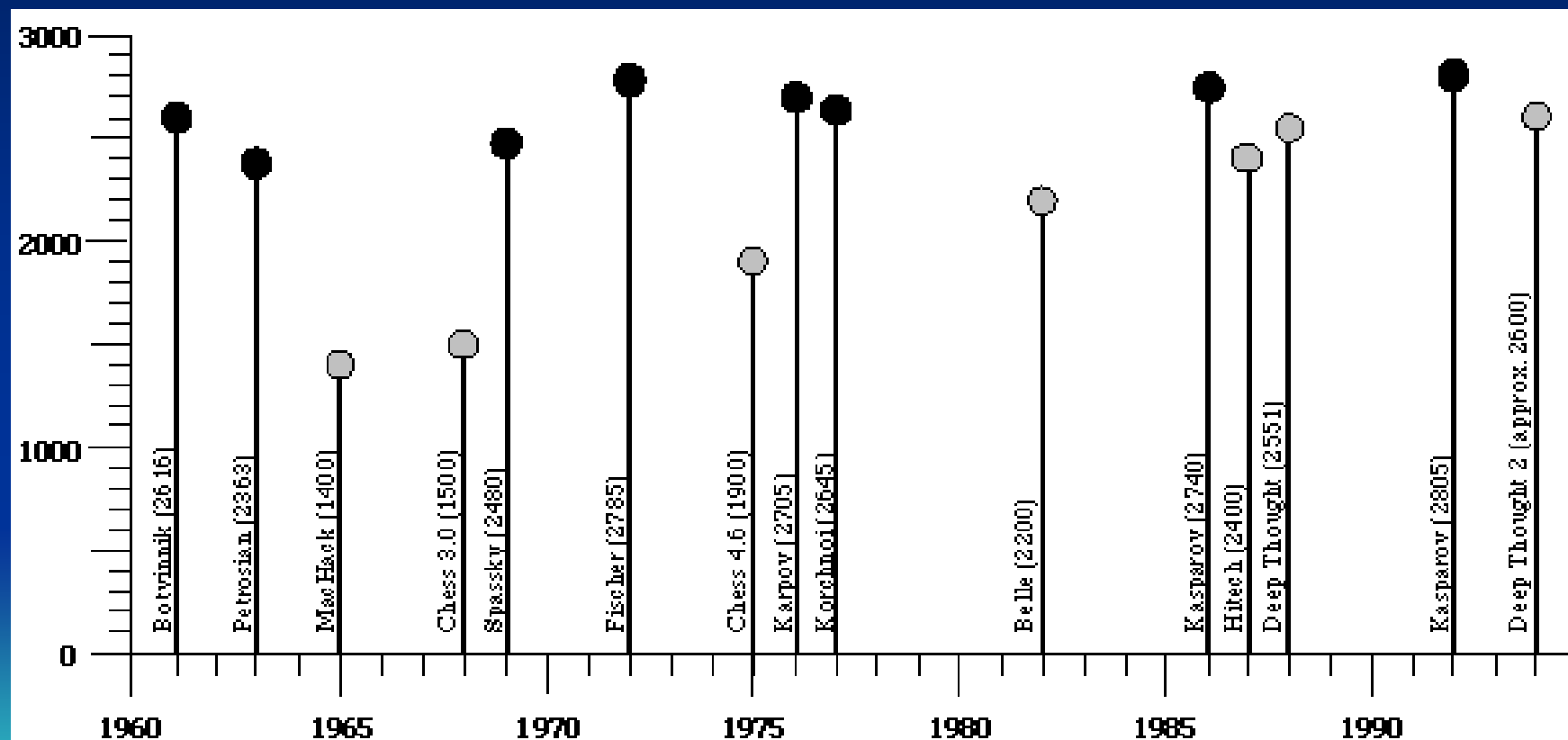


Meaning of the evaluation function



- Dealing with probabilities and expected values means we have to be careful about the “meaning” of values returned by the static evaluator.
- Note that a “relative-order preserving” change of the values would not change the decision of minimax, but could change the decision with chance nodes.
- Linear transformations are ok

Ratings of Human and Computer Chess Champions



may 11th

game 6: may 11 @ 3:00PM EDT | 19:00 GMT

kasparov 2.5 deep blue 2.5

[Home](#) [The match](#) [The players](#) [The technology](#) [Community](#)

Deep Blue Wins 3.5 to 2.5

KASPAROV vs DEEP BLUE
the rematch

With a dramatic victory in Game 6,
Deep Blue won its six-game rematch
with Champion Garry Kasparov

OVERVIEW

EVENT COVERAGE

MATCH NEWS

MAIN STORIES



Commentary

George Plimpton on chess, Kasparov, and the limitations of computers

[Read the article](#)



Commentary

Vishwanathan Anand on the legacy of Kasparov vs. Deep Blue

[Read the article](#)



Club Kasparov

Visit the virtual home of the world's greatest chess player.



Guest essays

Thoughts on chess, computers, and what it all means

[Read the essays...](#)



Community

During the rematch, more than 20,000 people from 120 countries joined the community to talk about the match.



Clips from the rematch

Video footage from the games

[Highlights from the games](#)

[Press room](#) [Chess reference](#) [Feedback](#) [Site guide](#)



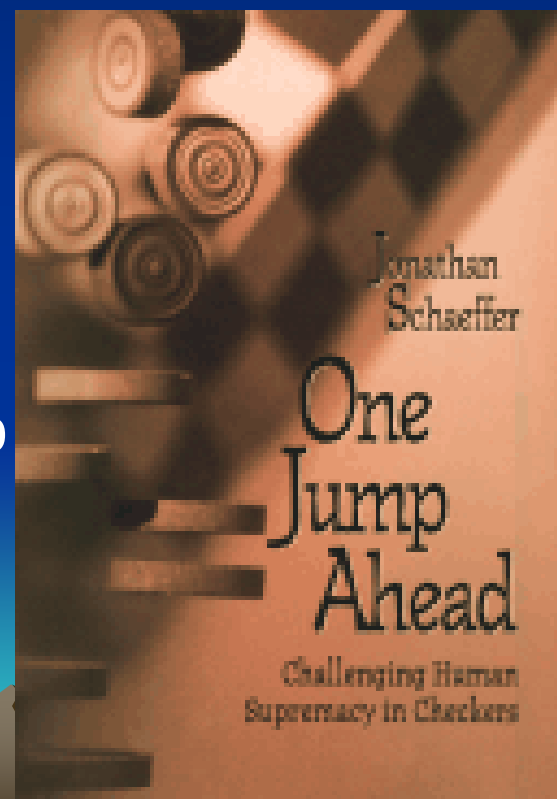
Chinook

- Chinook is the World Man-Machine Checkers Champion developed by researchers at the University of Alberta.
- It earned this title by competing in human tournaments, winning the right to play for the (human) world championship, and eventually defeating the best players in the world.
- Visit
<<http://www.cs.ualberta.ca/~chinook/>> to play Chinook over the Internet.
- Read “One Jump Ahead: Challenging Human Supremacy in Checkers”
Jonathan Schaeffer, University of Alberta
(496 pages, Springer. \$34.95, 1998).

The board set for play



Red to play



Assignment

- Read the supplied soft copy on CSP

